MAJOR PROJECT REPORT

Programming in 

Title of the Project:

''ENCRYPTION – DECRYPTION APPLICATION USING C''

Submitted By:

NAME: K Arya

SAP ID: 590025917

Batch: 38

B.Tech CSE

Submitted To:

Dr. Tanu Singh

Faculty, Programming in C

Academic Year: 2025.

DECLARATION

I, K Arya (SAP ID 590025917), hereby declare that the Major Project titled "Encryption–Decryption Application Using C Programming" has been completed by me under the guidance of Dr. Tanu Singh, School of Computer Science, UPES.

This project is my original work and has not been submitted elsewhere.
- K Arya
Date: 30TH NOVEMBER 2025.

ACKNOWLEDGEMENT

I would like to express my sincere thanks to Dr. Tanu Singh for guiding me throughout this project. The encouragement and suggestions I received really helped me in understanding the concepts of C programming better.

I also want to thank my batchmates and classmates who helped me test the program during development. This project improved my understanding of strings, functions, and modular programming.

— K Arya

TABLE OF CONTENTS

1. Abstract

This project focuses on creating a simple, menu-driven Encryption–Decryption Application using the C programming language. The aim was to convert normal text into encoded form and decode it back using three classical algorithms. The project helped me understand strings, loops, functions, multiple file structure and safe input handling.

The program compiles successfully using GCC and follows the exact folder structure and GitHub submission guidelines provided for the Major Project in Programming in C.

---

2. Introduction

Encryption is one of the basic techniques used to protect data. Even though advanced cryptography exists today, classical ciphers are still very useful for understanding how encryption works at a fundamental level.
In this project, I implemented three such algorithms in C:

- **Caesar Shift Cipher**
- **Reverse + Shift Cipher**
- **Vigenere-style Cipher**

The application runs entirely in the terminal and uses modular coding practices.

---

3. Problem Definition

The goal was to build a C-program-based system where a user could:

- **Enter a message**
- **Choose from different encryption algorithms**
- **Encrypt the message**
- **Decrypt the message back**
- **Use simple keys (numeric or word key)**
- **Avoid runtime crashes due to wrong input**

The project also required following a specific GitHub folder structure and proper documentation.

---

4. Objectives

- **To apply concepts of C programming in a practical application**
- **To implement multiple encryption algorithms**
- **To use modular programming with multiple .c and .h files**
- **To understand input validation and string manipulation**
- **To follow standardized coding and documentation guidelines**
- **To use GitHub for proper version control and submission**

---

5. System Design

5.1 Overall Workflow

The program repeatedly shows a menu:

1. **Encrypt Text**
2. **Decrypt Text**
3. **Exit**

Depending on the user's selection, the program:

- **Asks the user to choose an algorithm**
- **Takes input text**
- **Takes a numeric or word key**
- **Applies the selected encryption/decryption logic**
- **Shows clear output**

The loop continues until the user exits.

5.2 Algorithms Used

Algorithm 1: Caesar Shift Cipher

Shifts each letter forward by a key value.
If the shift crosses ‘z’, it wraps back to ‘a’.
Non-alphabetic characters remain unchanged.

Algorithm 2: Reverse + Shift Cipher

First reverses the string, then shifts each character.
This makes the output much harder to guess.

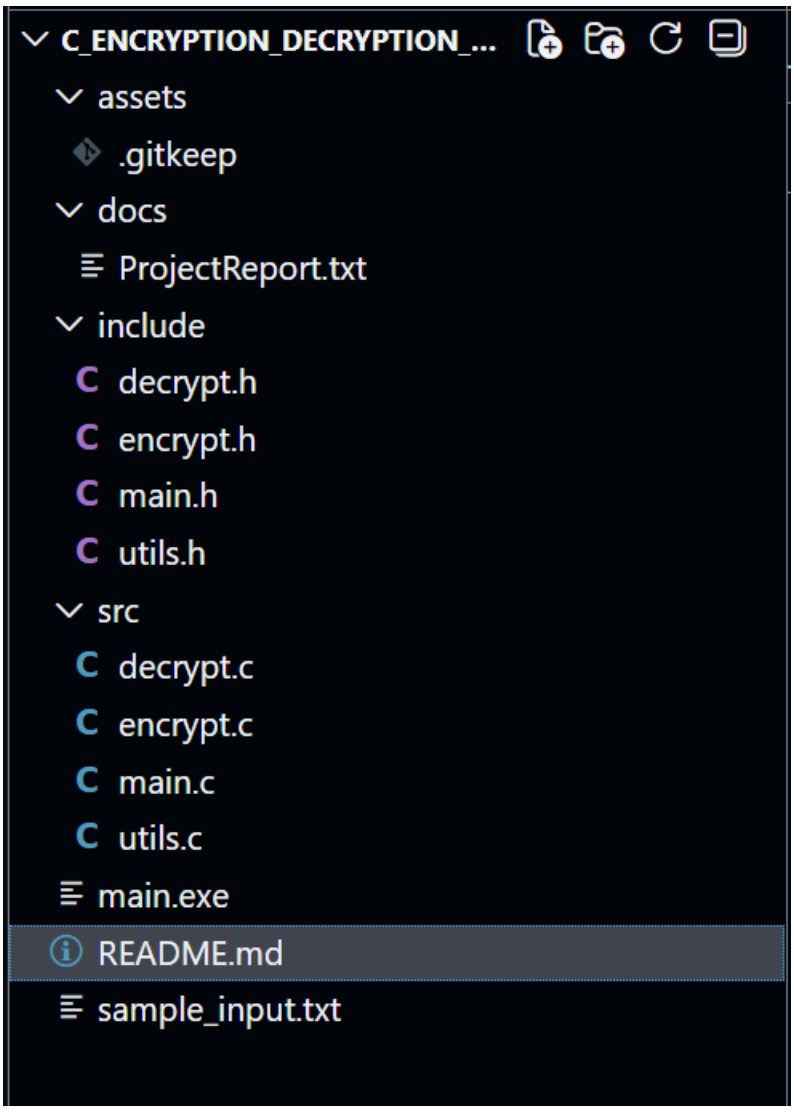
Algorithm 3: Vigenere-style Cipher

Uses a word as the key.
Each key character determines how much the input character should shift.
Key repeats automatically as needed.

6. Implementation Details

6.1 Modular Code Structure

The project is divided into multiple files to keep everything clean:



This approach makes the code easier to understand and maintain.

6.2 Code Snippets

Snippet 1 — Safe Integer Input

```
int read_int(void) {
  int value, result;
  while (1) {
    result = scanf("%d", &value);
    if (result == 1) {
      clear_input_buffer();
      return value;
    }
    printf("Invalid number. Try again: ");
    clear_input_buffer();
  }
}
```

Snippet 2 — Caesar Cipher Logic

```
char shift_char(char c, int key) {
  if (c >= 'A' && c <= 'Z')
    return (c - 'A' + key) % 26 + 'A';
  if (c >= 'a' && c <= 'z')
    return (c - 'a' + key) % 26 + 'a';
  return c;
}
```

Snippet 3 — Menu Loop

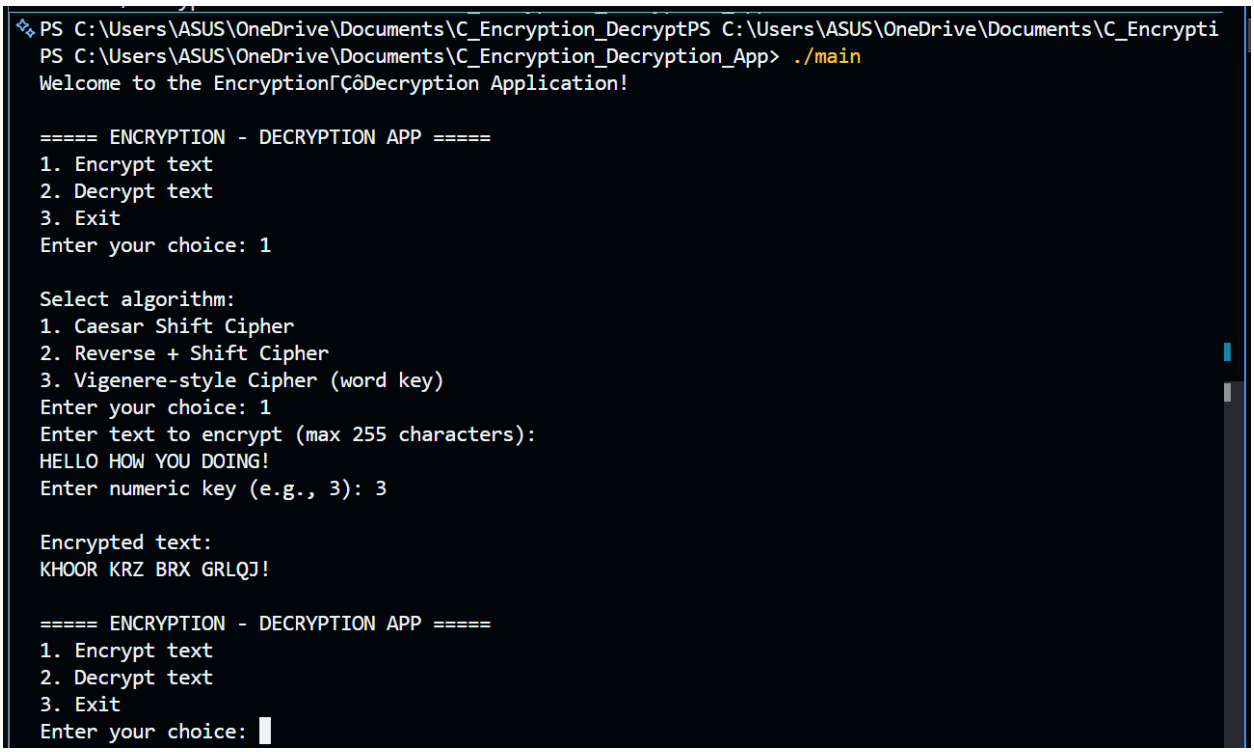
```
while (running) {
```

```
    show_main_menu();
    choice = read_int();

    if (choice == 1) { /* Encrypt */ }
    else if (choice == 2) { /* Decrypt */ }
    else if (choice == 3) running = 0;
    else printf("Invalid choice!\n");
}
```

7. Testing and Results

Figure 1: Output screenshot of program running (Encryption example).



8. Conclusion

This project helped me revise and apply major concepts taught in the Programming in C course. Implementing multiple ciphers improved my understanding of strings and character manipulation. Writing the program in a modular structure also made me more confident with header files and dividing logic properly. Overall, the project was a good learning experience and fulfills all required guidelines.

---

9. Future Scope

- **Add more cipher algorithms**
- **Add file encryption**
- **Build a simple GUI for non-technical users**
- **Implement key validation and stronger security**

---

10. References

1.Yashwant Kanetkar – *Let Us C*

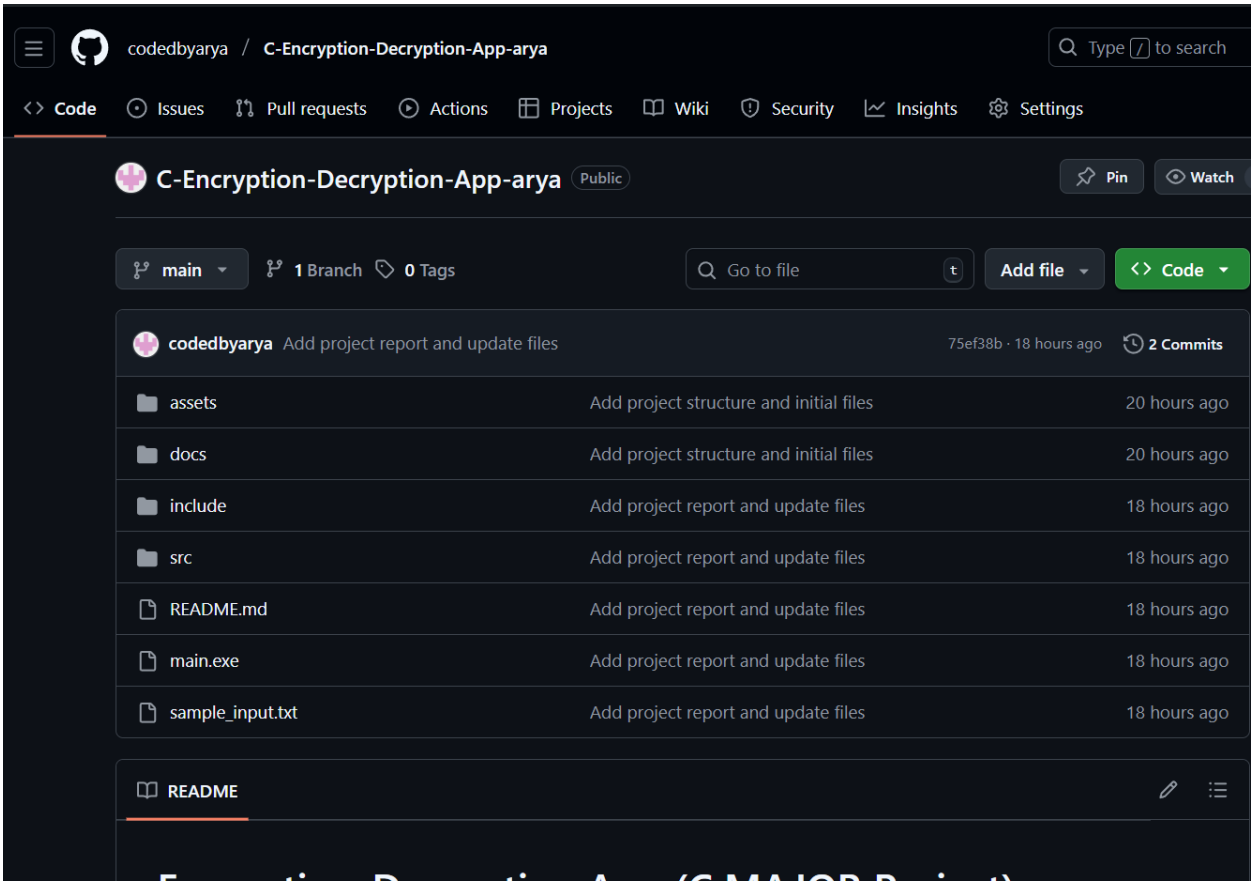
2.Classroom notes and lab content

3.Online references for Caesar/Vigenere ciphers.
4. Google browser.

---

11. APPENDIX

Figure 2: GitHub repository structure showing correct folder organization.



README

Encryption-Decryption App (C MAJOR Project)

This is my C programming major project for **CSEG1032 (Programming in C)**.
It is a menu-driven **Encryption–Decryption Application** that supports multiple algorithms.

🔹 Features

- Menu-based interface (Encrypt / Decrypt / Exit)
- Three different algorithms:
  i. **Caesar Shift Cipher**
  ii. **Reverse + Shift Cipher**
  iii. **Vigenere-style Cipher** using a word key
- Handles invalid numeric input safely
- Works on both uppercase and lowercase alphabets
- Clean modular code with separate **.c** and **.h** files

🔹 Folder Structure

```
/
├── src/
│   ├── main.c
│   ├── encrypt.c
│   ├── decrypt.c
│   └── utils.c
├── include/
│   ├── main.h
│   ├── encrypt.h
│   ├── decrypt.h
│   └── utils.h
├── docs/
│   └── ProjectReport.pdf   (final report will be placed here)
├── assets/
│   └── (screenshots / diagrams)
├── README.md
└── sample_input.txt
```

THANKYOU!!!!